



Gorgo Go for Java Programmers

May 31, 2026

Contents

3	Strings, Bytes, and Runes	1
	What a String Really Is	1
	byte and rune	1
	Indexing and Iteration	2
	String Literals	3
	Converting Between Strings, Bytes, and Runes	3
	The strings Package	3
	The strconv Package	5
	The unicode/utf8 Package	6
	The bytes Package	6
	Key Points	6
	Exercises	6

Chapter 3

Strings, Bytes, and Runes

Go strings look familiar — you create them with double quotes, concatenate with `+`, and pass them to `fmt.Println`. But the model underneath is different enough from Java's that it causes real bugs, especially with non-ASCII text. This chapter covers what Go strings actually are, why `len(s)` might surprise you, and how to work with text correctly.

What a String Really Is

In Java, a `String` is a sequence of `char` values, where each `char` is a UTF-16 code unit. In Go, a string is an **immutable sequence of bytes**. That is the entire definition. Go makes no promises about encoding; by convention almost all Go source code and string data is UTF-8, but the type itself is just bytes.

Under the hood, a string value is a small struct: a pointer to read-only memory and a length. It is essentially a read-only `[]byte` without the capacity field.

`len(s)` returns the number of **bytes** in the string, not the number of characters.

```
s := "café"
fmt.Println(len(s)) // 5, not 4 --- é is two bytes (0xC3 0xA9)
```



Wut: If you are used to Java where `"café".length()` returns 4, note that Go's `len("café")` returns 5 because `é` is encoded as two bytes in UTF-8. While that might make Java seem enlightened, `"👉🔥".length()` in Java returns 4!

byte and rune

Go has no `char` type. Instead it has two types for working with individual pieces of text:

- `byte` is an alias for `uint8`. It holds a single ASCII character or one byte of a multibyte sequence.
- `rune` is an alias for `int32`. It holds a full Unicode code point.

Java's `char` is a UTF-16 **code unit**, which means characters outside the Basic Multilingual Plane (anything above `U+FFFF`) require two Java `char` values. Go's `rune` is a full code point, so one `rune` always represents one character, no matter how far up the Unicode table it lives.

Rune literals use single quotes, just like character literals in Java:

```
var b byte = 'A' // 65
var r rune = '👉' // 8984 (U+2318 PLACE OF INTEREST SIGN)
var r2 rune = 'é' // 233 (U+00E9)
```

A rune literal is an untyped constant of kind `rune` (underlying type `int32`). Without an explicit type annotation it behaves as an untyped integer constant.

Indexing and Iteration

Indexing with `s[i]`

Indexing a string with `s[i]` gives you a **byte**, not a character.

```
s := "café"
fmt.Println(s[3]) // 195 (0xC3 --- the first byte of é)
```

Java programmers often expect `s[3]` to yield `'é'`. In Go it yields 195, the numeric value of the first byte of the two-byte UTF-8 encoding of `é`. Formatting it with `%c` would print `Ã`, which is the Latin character whose code point is 195 — not what you wanted.



Trap: `s[i]` gives you a byte. If the string contains any non-ASCII characters, walking it with a plain index loop and printing or storing individual indexed values will corrupt multibyte characters.

Iterating Bytes with a Plain for Loop

A plain index loop iterates bytes:

```
s := "café"
for i := 0; i < len(s); i++ {
    fmt.Printf("s[%d] = %d\n", i, s[i])
}
// s[0] = 99 (c)
// s[1] = 97 (a)
// s[2] = 102 (f)
// s[3] = 195 (first byte of é)
// s[4] = 169 (second byte of é, 0xA9)
```

Iterating Runes with `for range`

`for range` over a string decodes UTF-8 automatically. The loop variable receives a rune (the Unicode code point), and the index is the **byte position** of the start of that rune.

```
s := "café!"
for i, r := range s {
    fmt.Printf("s[%d] = %c (%d)\n", i, r, r)
}
// s[0] = c (99)
// s[1] = a (97)
// s[2] = f (102)
// s[3] = é (233)
// s[5] = ! (33)
```

Notice that the index jumps from 3 directly to 5 for the `!` — index 4 never appears because `é` occupies bytes 3 and 4, and byte 4 is not the start of a rune.



Tip: Use `for range` when you care about characters (runes). Use a plain `for i` loop when you need raw byte access.

String Literals

Go has two forms of string literal.

Interpreted string literals use double quotes and process backslash escapes — `\n`, `\t`, `\uXXXX`, `\UXXXXXXXX`, etc. This is the same as Java.

Raw string literals use backticks and suppress all escape processing. Everything between the backticks is literal, including newlines and backslashes.

```
// interpreted --- \n is a newline
msg := "Flowers\nfor you\n"

// raw --- \n is two characters: backslash and n
re := `d+\\.d+`

// raw --- multi-line JSON template
tpl := `{
    "artist": "Doja Cat",
    "album": "Scarlet"
}`
```

Raw string literals are especially useful for regular expressions (where backslashes are abundant) and for embedding multi-line text without escaping.

Converting Between Strings, Bytes, and Runes

You can convert freely among string, `[]byte`, and `[]rune`:

```
s := "hola"

b := []byte(s) // copy to a mutable byte slice
s2 := string(b) // copy back to a string

r := []rune(s) // copy to a rune slice
s3 := string(r) // copy back to a string

fmt.Println(s2) // hola
fmt.Println(s3) // hola
```



Wut: These conversions **copy** the data. A string and a `[]byte` pointing at the same bytes in Java is possible via `ByteBuffer` tricks, but in Go the compiler enforces the copy to maintain string immutability.

Converting a single integer to string gives you the UTF-8 encoding of that code point, not the decimal representation:

```
fmt.Println(string(rune(65))) // A
fmt.Println(string(rune(233))) // é
```

To convert a number to its decimal string representation, use `strconv.Itoa` (covered below).

The strings Package

The `strings` package provides the functions you reach for every day. Import it with `import "strings"`.

Searching and Testing

```
func Contains(s, substr string) bool // true if substr appears anywhere in s
func HasPrefix(s, prefix string) bool // true if s starts with prefix
func HasSuffix(s, suffix string) bool // true if s ends with suffix
func Count(s, substr string) int // number of non-overlapping occurrences of substr
func Index(s, substr string) int // byte index of first occurrence, or -1

s := "I can buy myself flowers"
fmt.Println(strings.Contains(s, "flowers")) // true
fmt.Println(strings.HasPrefix(s, "I can")) // true
fmt.Println(strings.HasSuffix(s, "flowers")) // true
fmt.Println(strings.Count(s, "l")) // 2
fmt.Println(strings.Index(s, "myself")) // 10
```

Splitting and Joining

```
func Split(s, sep string) []string // slice of substrings separated by sep
func Join(elems []string, sep string) string // concatenate elems with sep between each

parts := strings.Split("un,dos,tres", ",")
fmt.Println(parts) // [un dos tres]
fmt.Println(strings.Join(parts, " - ")) // un - dos - tres
```

Trimming and Case

```
func TrimSpace(s string) string // strip leading and trailing whitespace
func Trim(s, cutset string) string // strip any chars in cutset from both ends
func ToUpper(s string) string // return s in upper case
func ToLower(s string) string // return s in lower case
func ReplaceAll(s, old, new string) string // replace every occurrence of old with new
```

TrimSpace strips leading and trailing whitespace. Trim strips any characters in the cutset from both ends:

```
fmt.Println(strings.TrimSpace(" flores ")) // flores
fmt.Println(strings.Trim("***Espresso***", "*")) // Espresso
fmt.Println(strings.Trim("..Bad Habit..", "/.")) // Bad Habit
fmt.Println(strings.ToUpper("flowers")) // FLOWERS
fmt.Println(strings.ReplaceAll("la la la", "la", "na")) // na na na
```

Building Strings: strings.Builder

strings.Builder is the idiomatic way to build up a string incrementally. It is Go's answer to Java's StringBuilder. Unlike concatenating with + in a loop (which allocates a new string on every iteration), Builder maintains a growing buffer and materialises the final string only when you call String().

```
var b strings.Builder
b.WriteString("Espresso --- ")
b.WriteString("Sabrina Carpenter")
b.WriteByte('\n')
b.WriteRune('♪')
fmt.Println(b.String())
// Espresso --- Sabrina Carpenter
// ♪
```

The relevant methods are:

```

func (b *Builder) WriteString(s string) (int, error) // append a string
func (b *Builder) WriteByte(c byte) error          // append a single byte
func (b *Builder) WriteRune(r rune) (int, error)   // append a Unicode code point
func (b *Builder) String() string                  // return the accumulated string
func (b *Builder) Reset()                          // clear the buffer for reuse
func (b *Builder) Len() int                        // current length in bytes

```



Tip: Always use `strings.Builder` when you are building a string in a loop. Repeated `s += piece` is $O(n^2)$ in the total length because each `+` copies the entire accumulated string. `Builder` amortises this to $O(n)$.

The strconv Package

`strconv` handles conversions between strings and numeric types.

```

func Itoa(i int) string           // int → decimal string
func Atoi(s string) (int, error)  // decimal string → int
func FormatInt(i int64, base int) string // int64 → string in given base (2, 8, 10, 16)
func ParseInt(s string, base int, bitSize int) (int64, error) // string → int64; base 0 auto-detects
func FormatUint(i uint64, base int) string // uint64 → string in given base
func ParseUint(s string, base int, bitSize int) (uint64, error) // string → uint64
func FormatFloat(f float64, fmt byte, prec, bitSize int) string // float64 → string
func ParseFloat(s string, bitSize int) (float64, error) // string → float64
func FormatBool(b bool) string     // bool → "true"/"false"
func ParseBool(str string) (bool, error) // "true"/"false" → bool

```

`Itoa` and `Atoi` are convenient wrappers for base-10 `int`. When you need a specific base or size, reach for `ParseInt/ParseUint` and `FormatInt/FormatUint` directly:

```

strconv.FormatInt(255, 16) // "ff" --- hex
strconv.FormatInt(255, 2)  // "11111111" --- binary
strconv.ParseInt("ff", 16, 64) // 255, nil
strconv.ParseInt("0xFF", 0, 64) // 255, nil --- base 0 detects "0x" prefix
strconv.ParseUint("4294967295", 10, 32) // 4294967295, nil --- fits in uint32

```

The `bitSize` parameter (8, 16, 32, or 64) constrains the result range without changing the return type — `ParseInt` always returns `int64`, but passing `bitSize=32` guarantees the value fits in `int32`.

```

s := strconv.Itoa(42) // "42"
n, err := strconv.Atoi("99") // 99, nil
bad, err := strconv.Atoi("nope") // 0, *strconv.NumError

```

`strconv.Atoi` returns two values: the converted integer and an error. If the string is not a valid integer, `err` is non-`nil`. You should always check the error. [*no-discard-error*] Error handling is covered fully in Chapter 9; for now, the pattern is:

```

n, err := strconv.Atoi(s)
if err != nil {
    // handle the error
}

```



Trap: `fmt.Sprintf("%d", n)` converts an integer to a string and works, but it is significantly slower than `strconv.Itoa` because `fmt` uses reflection. Prefer `strconv` when performance matters.

The unicode/utf8 Package

When you need to work at the rune level without converting the whole string to `[]rune`, the `unicode/utf8` package gives you the tools.

```
func RuneCountInString(s string) int           // number of runes in s (not bytes)
func DecodeRuneInString(s string) (r rune, size int) // first rune and its byte width
func ValidString(s string) bool                // true if s is valid UTF-8

s := "café"
fmt.Println(len(s))           // 5 (bytes)
fmt.Println(utf8.RuneCountInString(s)) // 4 (runes)

r, size := utf8.DecodeRuneInString(s)
fmt.Printf("first rune: %c, size: %d\n", r, size) // first rune: c, size: 1

fmt.Println(utf8.ValidString(s))           // true
fmt.Println(utf8.ValidString("\xff\xfe")) // false
```

`DecodeRuneInString` is useful when you want to peel off one rune at a time from the front of a string without a full `for` range loop.

The bytes Package

The `bytes` package mirrors the `strings` package but operates on `[]byte` instead of `string`. Every function in `strings` that takes a `string` has a counterpart in `bytes` that takes `[]byte`. For example, `bytes.Contains`, `bytes.Split`, `bytes.Join`, `bytes.TrimSpace`.

When you are working with data that is already in a `[]byte` (reading from a file or network, for instance), using `bytes` functions avoids the copy that `string(b)` would require.

`bytes.Buffer` is the older alternative to `strings.Builder` and supports both reading and writing — it implements `io.Reader` and `io.Writer`, making it useful for testing code that writes to an `io.Writer`.

Key Points

- Go strings are immutable byte sequences; `len(s)` counts bytes, not characters.
- `byte` is `uint8`; `rune` is `int32` (a full Unicode code point).
- `s[i]` yields a byte; use `for` range to iterate runes.
- Java `char` is a UTF-16 code unit; Go `rune` is a full code point — they are different concepts.
- Raw string literals (backticks) pass through all characters literally, including backslashes and newlines.
- Converting between `string`, `[]byte`, and `[]rune` always copies data.
- `strings.Builder` is the idiomatic way to build strings in a loop; avoid `+=` in loops.
- `strconv.Atoi` returns an error; always check it.
- `utf8.RuneCountInString` gives the true character count; `len` gives the byte count.
- The `bytes` package mirrors `strings` for `[]byte` data.

Exercises

1. **Think about it:** Go strings are described as “immutable sequences of bytes.” Java strings are also immutable. Given that both languages have immutable strings, why does Go’s `for` range behave differently from Java’s enhanced `for` loop over a `String`? What would have to be true about the loop for both languages to give the same result?
2. **What does this print?**

```

package main

import "fmt"

func main() {
    s := "Beyoncé"
    fmt.Println(s[6])
}

```

3. **Calculation:** How many bytes does `len("Beyoncé")` return? How many runes does `utf8.RuneCountInString("Beyoncé")` return? (Hint: B, e, y, o, n, c are each one byte; é is two bytes.)
4. **Where is the bug?** The following function tries to reverse each byte's case by operating on raw bytes:

```

func swapCase(s string) string {
    b := []byte(s)
    for i := range b {
        switch {
        case b[i] >= 'A' && b[i] <= 'Z':
            b[i] += 32
        case b[i] >= 'a' && b[i] <= 'z':
            b[i] -= 32
        }
    }
    return string(b)
}

func main() {
    fmt.Println(swapCase("Héroé"))
}

```

é is encoded as bytes 0xC3 (195) and 0xA9 (169). H is 72 and e is 101. Trace through the loop byte by byte. What does the function actually print, and what is fundamentally wrong with this approach for non-ASCII strings?

5. **Write a program:** Write a function `reverseString(s string) string` that returns the string with its runes in reverse order. For example, `reverseString("café")` should return `"éfac"`. Test it with at least one string that contains a multibyte character to confirm it handles Unicode correctly. (Hint: convert to `[]rune` first.)

